

Universidad de Concepción

Facultad de Ingeniería

Departamento Ingeniería Civil Informática

Entregable 3: Tablas Hash

Asignatura: Estructuras de Datos

Grupo 10: Ariel Cisternas, Agustín Salgado, Ignacio Silva

Profesor: José Fuentes

Repositorio: <https://github.com/elamiql/Lab-3-ED.git>

Índice

1. Introducción	2
2. Descripción de las estructuras de datos comparadas	2
2.1. Tabla hash con hashing abierto	2
2.2. Tabla hash con hashing cerrado	2
2.3. <code>unordered_map</code> de la STL de C++	2
3. Descripción del dataset y del ambiente de experimentación	3
4. Resultados experimentales	3
4.1. Tamaño de las estructuras de datos	3
4.2. Rendimiento en la creación de la estructura	3
4.3. Clave más eficiente por estructura de datos	4
5. Conclusiones	4

1. Introducción

Este trabajo compara experimentalmente cinco implementaciones de tablas hash para contar cuántos tweets publicó cada usuario en un dataset de 180.000 tweets sobre las elecciones australianas de 2019.

Las estructuras evaluadas son una tabla hash con hashing abierto, tres variantes de tabla hash con hashing cerrado y `unordered_map` de la STL de C++. Cada estructura se probó usando dos claves distintas: `user_id` y `user_screen_name`, con el fin de evaluar si el tipo de clave afecta el rendimiento.

El experimento se implementó en C++20 y se compiló con `g++`. Se midió el tiempo de creación acumulado en 18 checkpoints y el tamaño en memoria de la estructura resultante, repitiendo cada medición 20 veces para reportar promedio y desviación estándar.

Como conclusión preliminar, las estructuras con hashing cerrado resultaron entre 5 y 6 veces más rápidas que el encadenamiento al usar `user_id` como clave. Por otro lado, `unordered_map` fue la opción más rápida con `user_screen_name`. El encadenamiento fue siempre la estructura más lenta, presentando una brecha de rendimiento mucho mayor con claves enteras que con claves de texto.

2. Descripción de las estructuras de datos comparadas

El código fuente completo se entrega adjunto en el archivo `código_fuente.zip`, documentado con comentarios. Además, se adjunta el enlace al repositorio de GitHub en la portada.

2.1. Tabla hash con hashing abierto

Cada bucket es una lista enlazada de pares [1]. El costo promedio de inserción y búsqueda es $O(1 + \alpha)$, donde α es el factor de carga, mientras que el peor caso es $O(n)$. Se usa un arreglo inicial de 1024 buckets con un factor de carga máximo de 1.0. Al superar este límite se realiza un rehash duplicando la cantidad de buckets. La función hash primaria es multiplicativa para `user_id` y FNV-1a de 64 bits para `user_screen_name` [2].

2.2. Tabla hash con hashing cerrado

Las tres variantes comparten la misma implementación de slots contiguos [1], variando solo la función de sondeo: *linear probing* usa $(h_1 + i)$ mód cap, *quadratic probing* usa $(h_1 + i^2)$ mód cap y *double hashing* usa $(h_1 + i \cdot h_2)$ mód cap. El costo promedio de inserción y búsqueda es $O(1/(1 - \alpha))$ y el peor caso es $O(n)$. La capacidad inicial es 1024, ajustada siempre al número primo más cercano con un factor de carga máximo de 0.7. Al superar este límite se duplica la capacidad y se busca el siguiente primo. Las funciones hash primarias son las mismas del encadenamiento y la secundaria es una variante `djb2` acotada al rango [1, 97] [3].

2.3. `unordered_map` de la STL de C++

Implementación de la biblioteca estándar [4], basada internamente en encadenamiento con administración automática de buckets y rehash según su factor de carga máximo por defecto. El costo promedio de inserción y búsqueda es $O(1)$ y el peor caso es $O(n)$. Se usó `reserve(1024)` al crear cada tabla para iniciar con una capacidad comparable a las estructuras propias, utilizando las funciones hash por defecto de la biblioteca para enteros y cadenas de texto.

3. Descripción del dataset y del ambiente de experimentación

El dataset corresponde a 180.000 tweets publicados en mayo de 2019 sobre las elecciones australianas, en formato CSV. De sus 11 columnas solo se usaron `user_id` y `user_screen_name`. El archivo se lee con un parser CSV propio implementado en C++ que respeta comillas dobles, comas y saltos de línea dentro de campos citados. Las filas cuyo `user_id` no pudo convertirse a entero se descartan silenciosamente. No se realizó ninguna otra modificación al dataset.

Los experimentos se ejecutaron en un notebook Lenovo con procesador Intel Core i5-10300H de 4 núcleos y 8 hilos, y 16 GB de RAM. El procesador cuenta con caché L1 de 32 KB, caché L2 de 256 KB por núcleo y caché L3 compartida de 8 MB. El sistema operativo es Windows y la compilación se realizó con `g++` usando las banderas `-std=c++20 -O2`. Durante las mediciones se cerraron todos los programas no esenciales, dejando únicamente el ejecutable de los experimentos corriendo.

Cada una de las 10 configuraciones se repitió 20 veces, midiendo el tiempo acumulado en 18 checkpoints y registrando 3.600 mediciones individuales en el archivo `benchmark_results.csv`.

4. Resultados experimentales

4.1. Tamaño de las estructuras de datos

La Figura 1 muestra el tamaño en memoria de cada estructura al finalizar la carga completa. Las tres variantes de hashing cerrado ocupan exactamente el mismo espacio porque comparten la misma política de capacidad y tamaño de slot. El encadenamiento y `unordered_map` ocupan más memoria debido al uso de nodos de lista enlazada.

Figura 1: Tamaño en memoria de cada estructura en el checkpoint final (180.000 tweets), por tipo de clave.

Cuadro 1: Tamaño en memoria al finalizar la carga de 180.000 tweets.

Estructura	<code>user_id</code> (MB)	<code>screen_name</code> (MB)
Chaining (encadenamiento)	1.70	2.73
Open addressing - linear	1.01	2.52
Open addressing - quadratic	1.01	2.52
Open addressing - double hashing	1.01	2.52
<code>unordered_map</code> (STL)	1.80	2.83

4.2. Rendimiento en la creación de la estructura

La Figura 2 muestra el tiempo promedio de creación en función de la cantidad de tweets procesados. En el gráfico de *chaining* se aprecia un salto en la curva de `user_screen_name` cerca de los 90.000 tweets, el cual corresponde a un rehash de la tabla.

Figura 2: Tiempo promedio de creación (ms) vs. tweets almacenados, por estructura, colisión y tipo de clave.

La Tabla 2 resume el tiempo promedio y la desviación estándar en el checkpoint final para las 20 repeticiones de cada configuración:

Cuadro 2: Tiempo promedio y desviación estándar en el checkpoint final.

Estructura	user_id	screen_name
	media $\pm \sigma$ (ms)	media $\pm \sigma$ (ms)
Chaining (encadenamiento)	48,17 $\pm$ 1,09	22,41 $\pm$ 0,50
Open addressing - linear	7,78 $\pm$ 0,11	15,40 $\pm$ 0,45
Open addressing - quadratic	7,61 $\pm$ 0,17	14,32 $\pm$ 0,31
Open addressing - double hashing	7,84 $\pm$ 0,15	16,89 $\pm$ 0,37
unordered_map (STL)	8,70 $\pm$ 0,15	13,79 $\pm$ 0,37

4.3. Clave más eficiente por estructura de datos

Al comparar ambas claves en el checkpoint final, se observa que en cuatro de las cinco estructuras `user_id` es más rápido que `user_screen_name`. La única excepción es el encadenamiento, donde ocurre lo contrario.

Cuadro 3: Comparación de eficiencia según tipo de clave.

Estructura	Clave más eficiente	Ganador (ms)	Perdedor (ms)
Chaining (encadenamiento)	user_screen_name	22.41	48.17 (user_id)
Open addressing - linear	user_id	7.78	15.40 (screen_name)
Open addressing - quadratic	user_id	7.61	14.32 (screen_name)
Open addressing - double hashing	user_id	7.84	16.89 (screen_name)
unordered_map (STL)	user_id	8.70	13.79 (screen_name)

Esta anomalía se explica por la relación entre la capacidad de la tabla y su función hash. En el encadenamiento la cantidad de buckets siempre es una potencia de 2 y la función hash de `user_id` es multiplicativa. Al tomar el módulo por una potencia de 2 solo se conservan los bits menos significativos del producto, los cuales distribuyen mal y generan un exceso de colisiones. Las estructuras de hashing cerrado no sufren este problema porque su capacidad siempre es un número primo.

5. Conclusiones

Para la clave `user_id`, las tres variantes de hashing cerrado fueron las más rápidas, seguidas por `unordered_map`. El encadenamiento fue la estructura más lenta. Para la clave `user_screen_name`, `unordered_map` fue la opción más rápida, seguida de *quadratic probing*, *linear probing* y *double hashing*. El encadenamiento volvió a ser el más lento, pero con una diferencia de tiempo mucho menor que con la clave entera.

Respecto al tipo de clave, `user_id` fue más eficiente que `user_screen_name` en cuatro de las cinco estructuras evaluadas. Esto es coherente, ya que procesar y comparar enteros es más rápido que operar con cadenas de texto. La única excepción fue el encadenamiento, comportamiento causado por la mala distribución de la función hash multiplicativa al usar una tabla con tamaño potencia de 2.

En cuanto a la memoria, las tres variantes de hashing cerrado fueron las más compactas. Por el contrario, `unordered_map` fue la estructura más pesada debido al uso de punteros en sus nodos internos. En síntesis, considerando velocidad y eficiencia de memoria, *quadratic probing* ofrece el mejor equilibrio general y es altamente competitivo con ambos tipos de clave.

Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.^a ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] D. Eastlake, T. Hansen, G. Fowler, K.-P. Vo y L. Noll, “The FNV Non-Cryptographic Hash Algorithm”, IETF, Internet-Draft draft-eastlake-fnv-17, 2019.
- [3] D. J. Bernstein, “djb2 Hash Algorithm”, *comp.lang.c*, 1991. [En línea]. Disponible en: <http://cr.yp.to/djb.html>
- [4] ISO/IEC, “Programming Languages C++”, Estándar ISO/IEC 14882:2020 (C++20), 2020.